FAST NUCES — Karachi Campus

AI Department

University Room Booking System

Milestone 1 — Project Proposal & Planning

Fundamentals of Software Engineering

Prepared by

Abdullah Sonija — 24K-0013

Zahid Hussain — 24K-0015

Date: 13 March 2026

1. Executive Summary

The University Room Booking System (URBS) is a web-based application designed to streamline the process of reserving university rooms for students, faculty, and administrative staff. The system replaces inefficient manual booking processes — such as email chains, phone calls, and paper-based records — with a centralized, automated digital platform.

The proposed system enables students and faculty to search for available rooms, submit booking requests, and track reservation status in real time. Administrators are provided with a dedicated dashboard to manage room inventory, approve or reject booking requests, and monitor overall room utilization.

This document serves as the formal project proposal for Milestone 1, outlining the system goals, scope, stakeholders, process model, team responsibilities, project timeline, and identified risks. The project will be executed over six weeks by a two-member team following an Agile/Scrum development methodology.

2. Problem Statement

Universities face persistent challenges in managing physical room resources efficiently. The following core problems motivate the development of URBS:

Problem	Real-World Impact
Double Booking	Two groups reserve the same room simultaneously, causing conflict and disruption
Manual Approval	Administrative staff spend excessive time processing booking requests manually via email
Poor Record Keeping	Historical room usage data is difficult to retrieve, making planning and auditing nearly impossible
Scheduling Conflicts	Overlapping reservations by different departments frequently lead to operational disruptions

3. Project Objectives

The primary objectives of the University Room Booking System are:

- Develop a role-based web application supporting three distinct user types: Student, Faculty, and Admin

- Provide a real-time room availability search with filters for date, time, room type, and capacity
- Implement an automated booking workflow that eliminates manual email-based processes
- Enforce conflict prevention at both the application and database level to prevent double-booking
- Deliver an admin dashboard for room management, booking approval, and reservation oversight
- Ensure complete input validation across all forms in compliance with course requirements
- Follow the full Software Development Life Cycle (SDLC) with documented artifacts for each phase
- Produce a working Minimum Viable Product (MVP) demonstrable to real university users

4. Project Scope

4.1 In Scope

- User registration and login with role-based access control (Student, Faculty, Admin)
- Room browsing and availability search by date, time slot, type, and capacity
- Booking request submission with purpose, time, and date selection
- Admin approval and rejection workflow with remarks
- Double-booking conflict detection and prevention
- User booking history and status tracking
- Admin room management (add, edit, delete rooms)
- Input validation on all forms (frontend and backend)
- Responsive web UI accessible on desktop and mobile browsers

4.2 Out of Scope

- Mobile native application (iOS / Android)
- Payment or fee processing for room reservations
- Integration with external university ERP or HRMS systems
- Cloud deployment (project will be demonstrated locally)
- Real-time push notifications via SMS or email (marked as optional)

5. Stakeholders & System Actors

The following individuals and groups have a direct interest in or interact with the University Room Booking System:

Actor	Type	Role & Interaction
Student	Primary User	Searches rooms, submits booking requests, views booking history
Faculty	Primary User	Books rooms for lectures, meetings, or research sessions with priority access
Admin	System Manager	Manages room inventory, approves or rejects requests, views full schedule
System	Automated Actor	Validates bookings, detects conflicts, stores reservation records
Course Instructor	Stakeholder	Evaluates project artifacts, documentation, and final demonstration

6. Proposed Process Model — Agile / Scrum

6.1 Model Selection & Justification

The project will follow the Agile methodology with Scrum as the project management framework. This selection is justified by the following reasons:

- The project has evolving requirements — user feedback in Milestone 4 may change features
- Short 2-week sprints map perfectly onto the milestone deadlines defined by the course
- A 2-person team is ideally suited for lightweight Scrum without overhead
- Scrum artifacts (backlog, sprint reviews, retrospectives) serve as direct evidence for Milestone 7
- Agile allows early delivery of a working prototype (Milestone 4 alpha release)
- The course rubric explicitly lists Agile/Scrum as expected methodologies

6.2 Scrum Configuration

Scrum Element	Detail
Sprint Length	2 weeks per sprint
Scrum Master	Abdullah Sonija (rotates per sprint)
Product Owner	Both members jointly (instructor as stakeholder)
Backlog Tool	Trello board with three columns: To Do / In Progress / Done

Scrum Element	Detail
Daily Standup	10-minute check-in covering: what was done, what is planned, any blockers
Sprint Review	End-of-sprint demo of completed functionality
Retrospective	Document what worked well, what to improve, and action items
Version Control	GitHub repository with branch-per-feature strategy

7. Team Roles & Responsibilities

Team Member	Primary Role	Responsibilities
Abdullah Sonija 24K-0013	Backend Developer & Project Lead	Flask REST API development, Database schema & models, JWT Authentication & authorization, Business logic & conflict detection, Sprint planning & coordination
Zahid Hussain 24K-0015	Frontend Developer & Docs Lead	React.js UI development, API integration, UI wireframes & mockups, SRS documentation, UML diagram creation, Testing coordination

7.1 Shared Responsibilities

- Project proposal and planning (Milestone 1)
- Requirements gathering from real users
- System testing — unit, integration, and user acceptance testing
- Final project report and presentation
- Project evaluation and retrospective (Milestones 6 & 7)

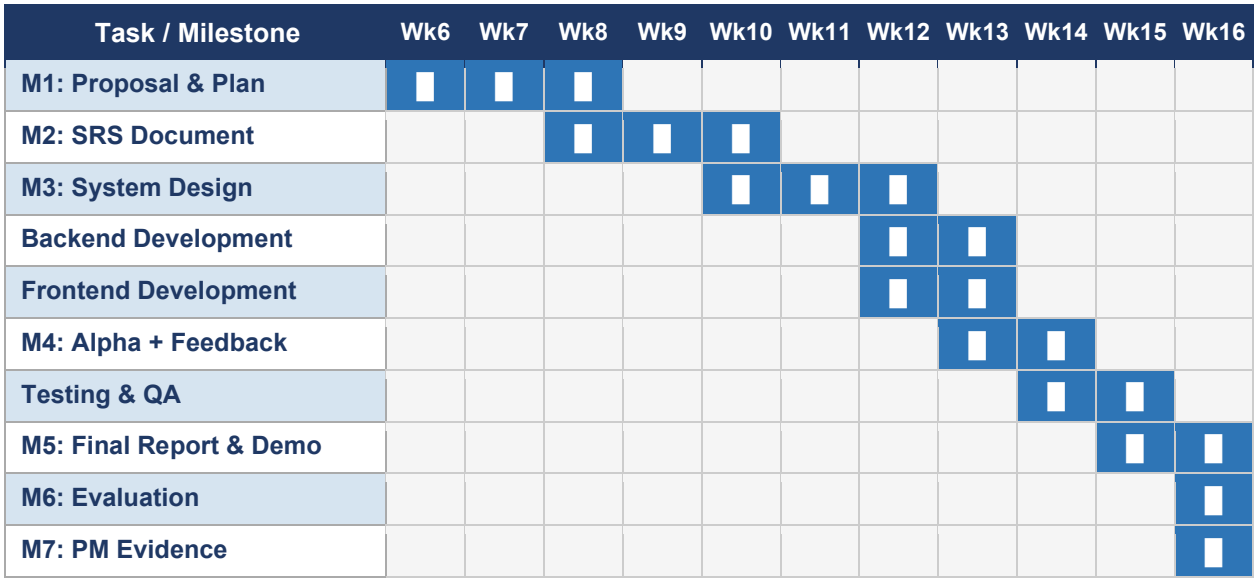
8. Task Breakdown by Sprint

Sprint	Tasks	Abdullah (24K-0013)	Zahid (24K-0015)
Sprint 1 (Wk 6–8)	Project Proposal, Gantt Chart, Scrum Setup, Risk Register	Risk analysis, Tech stack, Planning	Proposal writing, Document formatting

Sprint	Tasks	Abdullah (24K-0013)	Zahid (24K-0015)
Sprint 2 (Wk 8–10)	Requirements gathering, SRS document, Use Case diagrams	Functional requirements, ER diagram	SRS writing, Use Case & UML diagrams
Sprint 3 (Wk 10–12)	System architecture, DB design, wireframes, all UML	DB schema, API design, deployment diagram	Wireframes, Class/Sequence/Activity diagrams
Sprint 4 (Wk 12–13)	Backend development: APIs, auth, database, conflict logic	All Flask endpoints, JWT, SQLAlchemy models	React app setup, routing, component structure
Sprint 5 (Wk 13–14)	Frontend development, API integration, alpha release	Bug fixes, backend refinement	All UI screens, API integration, alpha demo
Sprint 6 (Wk 14–16)	Feedback iteration, final testing, report, presentation	Final fixes, code documentation	Final report, presentation slides, evaluation

9. Project Timeline — Gantt Chart

The following Gantt chart illustrates the planned schedule across 11 weeks (Week 6 to Week 16):



10. Technology Stack

Layer	Technology	Purpose
Frontend	React.js + Tailwind CSS	User interface, responsive design, dynamic rendering
Backend	Python Flask + Flask-RESTful	REST API, business logic, server-side validation
Database	SQLite (dev) / MySQL (prod)	Persistent storage, relational data, conflict constraints
Authentication	JWT (Flask-JWT-Extended)	Secure token-based auth, role enforcement per endpoint
ORM	SQLAlchemy	Database abstraction, model relationships
Version Control	Git + GitHub	Source control, collaboration, milestone tracking
Project Mgmt	Trello	Scrum board, sprint backlog, task tracking
Diagrams	Draw.io / Lucidchart	UML, ER diagram, architecture, wireframes

11. Risk Register

Risk	Likelihood	Impact	Mitigation Strategy
Double booking race condition	Medium	High	Enforce UNIQUE constraint on (room_id, date, start_time) at DB level
Scope creep with 2-person team	High	High	Strict MVP scope defined upfront; new features logged but not implemented until core is done
Unauthorized API access	Medium	High	JWT middleware validates every protected endpoint; role checked server-side before any action
React/Flask integration errors	Medium	Medium	Use CORS configuration on Flask; test each API endpoint independently before integrating
Data loss during development	Low	Medium	Regular commits to GitHub; maintain a seed script to restore test data

Risk	Likelihood	Impact	Mitigation Strategy
Team member unavailability	Low	High	All code is on GitHub and documented; any member can continue the other's work
Poor user feedback quality	Medium	Low	Prepare structured feedback forms with specific questions; test with at least 3 real users

12. Deliverables by Milestone

Milestone	Week	Deliverables
M1	Week 8	Project Proposal document, Gantt chart, risk register, Scrum setup
M2	Week 10	SRS document (IEEE 830 format), Use Case diagrams, FR & NFR tables
M3	Week 12	Architecture diagram, ER diagram, Class diagram, Sequence diagram, Activity diagram, Wireframes
M4	Week 14	Working alpha application, user feedback report, change log
M5	Week 16	Final polished application, final report, presentation slides, code repository
M6	Week 16	Project evaluation report — strengths, weaknesses, lessons learned
M7	Week 16	Trello board screenshots, sprint retrospective notes, GitHub commit history, task allocation log

13. Conclusion

The University Room Booking System addresses a real and widely experienced problem in academic institutions. By automating the room reservation process with a role-based web application, URBS will eliminate scheduling conflicts, improve transparency, and reduce the administrative burden on university staff.

The two-member development team will execute this project using the Agile/Scrum methodology, ensuring iterative development with frequent reviews and course corrections. The technology stack

— React, Flask, and SQLite/MySQL — is lightweight and well-suited for delivering a functional MVP within the six-week development window.

This proposal establishes the foundation for all subsequent milestones. With a clearly defined scope, structured task breakdown, realistic timeline, and identified risk mitigations, the team is fully prepared to proceed with requirements gathering and the Software Requirements Specification (SRS) in Milestone 2.